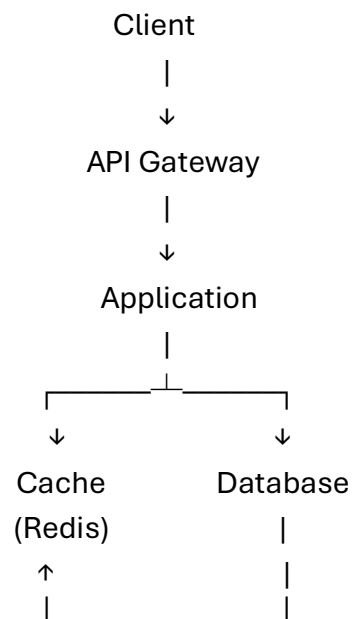


EPIC 07 Task 03 - Caching & Performance Architecture

Objective

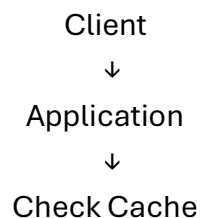
Design a caching system to improve application **performance**, reduce database load, and provide faster responses.

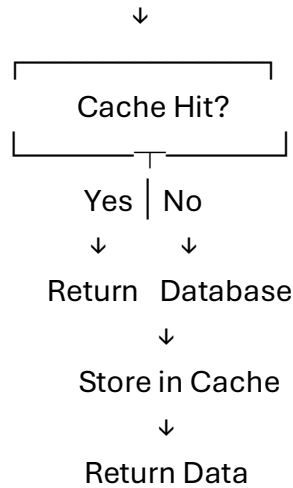
Architecture



1. Cache-Aside

In **cache-aside**, the application checks the cache first.





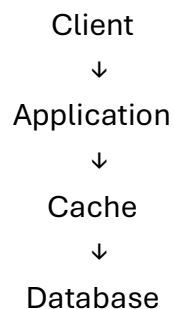
Example

If a user requests product 101:

1. Application checks Redis.
2. If product 101 exists → return it from cache.
3. If it does not exist → read from database.
4. Store the results in Redis.
5. Return the product to the client.

2. Read-Through Cache

With **read-through caching**, the application reads through the cache layer.

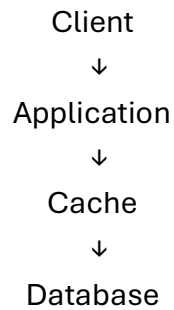


If data is not available in the cache, the cache automatically obtains it from the database and stores it.

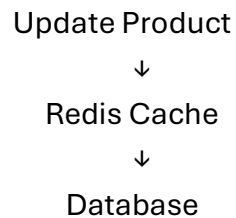
Benefit: Simplifies repeated read operations.

3. Write-Through Cache

In **write-through caching**, data is written to the cache and database together.



Example:



This helps keep cached data consistent with the database.

4. Cache Invalidation

Cache invalidation means removing or updating old data from the cache when the original data changes.

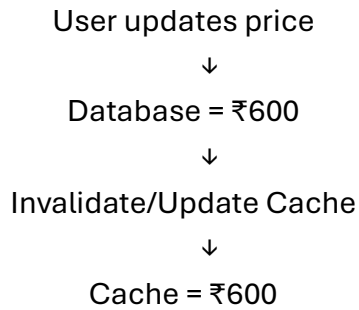
Example:

Database:

Product 101 = ₹500

Cache:

Product 101 = ₹500



Common strategies:

- **TTL (Time to Live)** — automatically expires data after a specified time.
- **Delete on update** — remove cached data when database data changes.
- **Update on write** — update cache whenever database data is updated.

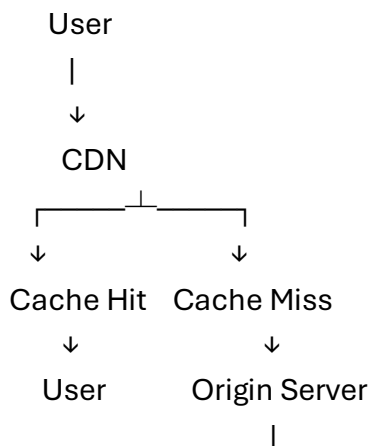
5. CDN Basics

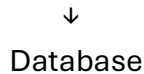
A **CDN (Content Delivery Network)** stores static content closer to users.

Examples of content:

- Images
- Videos
- CSS
- JavaScript
- Static web pages

Architecture





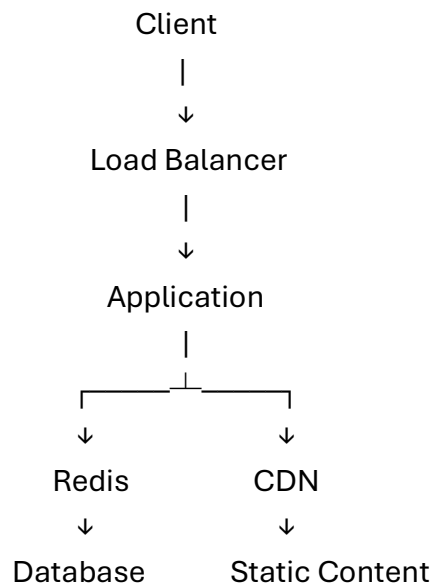
For example, if an image is already available at the CDN edge server, the user receives it without requesting it from the main application server.

6. Performance Improvements

Caching improves performance by:

- Reducing database queries.
- Reducing response time.
- Handling more concurrent users.
- Reducing load on application servers.
- Improving scalability.

Final Design



Conclusion: The caching architecture uses **cache-aside, read-through, write-through, cache invalidation, and CDN concepts** to reduce database load and improve application performance.